

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA1401

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom-up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 10%

QUESTIONS: 3

**Duration :60 Mins
Off Class
Total Marks:30**

Annexure A

SCENARIO BASED TASK

Code of Conduct:

I Enamala Gowtham /192311190_(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Enamala Gowtham

S.NO:10 RegNo:192311190

Questions:

1. A language supports blocks with statements: Grammar:

$S \rightarrow \text{begin } L \text{ end } \mid L \rightarrow S ; L \mid S$

$S \rightarrow \text{id}$ Input:

begin id ; id end

Tasks:

- (i) Explain how sequence is handled
- (ii) Demonstrate parsing
- (iii) Construct parse tree
- (iv) Identify ambiguity
- (v) Modify grammar

Given Grammar

$S \rightarrow \text{begin } L \text{ end } \mid \text{id}$

$L \rightarrow S ; L \mid S$

Input:

begin id ; id end

(i) Explain how sequence is handled

The grammar represents a block of statements using the non-terminal L.

- $S \rightarrow \text{begin } L \text{ end}$ defines a block that starts with begin and ends with end.
- $L \rightarrow S ; L$ allows multiple statements separated by semicolons.
- $L \rightarrow S$ indicates the last statement in the sequence.

For the given input,

begin id ; id end

the sequence consists of two statements:

id

id

which are separated by a semicolon and enclosed within begin and end.

(ii) Demonstrate Parsing

Leftmost Derivation

S

$\Rightarrow \text{begin } L \text{ end}$

$\Rightarrow \text{begin } S ; L \text{ end}$

$\Rightarrow \text{begin id ; } L \text{ end}$

$\Rightarrow \text{begin id ; } S \text{ end}$

$\Rightarrow \text{begin id ; id end}$

Rightmost Derivation

S

$\Rightarrow \text{begin } L \text{ end}$

$\Rightarrow \text{begin } S ; L \text{ end}$

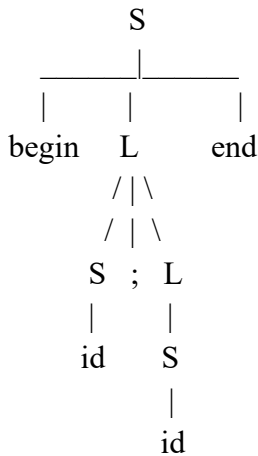
$\Rightarrow \text{begin } S ; S \text{ end}$

$\Rightarrow \text{begin } S ; \text{id end}$

$\Rightarrow \text{begin id ; id end}$

The input is successfully parsed according to the grammar.

iii) Construct Parse Tree



The parse tree shows that the block contains two identifier statements separated by a semicolon.

(iv) Identify Ambiguity

The given grammar is not ambiguous.

Reason:

- Every valid input has only one possible parse tree.
- The semicolon explicitly separates statements.
- The recursive production $L \rightarrow S ; L$ ensures a unique statement sequence.

Hence, the grammar produces only one valid derivation for the given input.

(v) Modify the Grammar

A simpler and more standard grammar for statement lists is:

$S \rightarrow \text{begin } L \text{ end} \mid \text{id}$

$L \rightarrow L ; S \mid S$

or in Extended Backus-Naur Form (EBNF):

$S \rightarrow \text{begin } L \text{ end} \mid \text{id}$

$L \rightarrow S ('; S)^*$

This grammar clearly represents one or more statements separated by semicolons while preserving the same language.

Conclusion:

The input

begin id ; id end

is successfully accepted by the grammar. The statement sequence is handled through the non-terminal **L**, parsing produces a unique parse tree, the grammar is unambiguous, and the modified grammar provides a clearer representation of statement lists.

2. A compiler supports modulus operator %. Grammar:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid T \% F \mid F \quad F \rightarrow \text{id}$

Input:

id + id % id * id

Tasks:

- (i) Explain precedence (*, % > +)
- (ii) Demonstrate parsing
- (iii) Show derivation
- (iv) Construct parse tree
- (v) Discuss ambiguity

Grammar:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid T \% F \mid F$

$F \rightarrow \text{id}$

Input:

id + id % id * id

(i) Explain Operator Precedence (*, % > +)

The grammar defines the precedence of operators using different non-terminals.

- E handles the addition (+) operator.
- T handles the multiplication (*) and modulus (%) operators.
- F represents an identifier (id).

Since * and % are derived from T while + is derived from E, multiplication and modulus have higher precedence than addition.

Therefore, the input is evaluated as:

id + ((id % id) * id)

(ii) Demonstrate Parsing

The parser first evaluates the higher-precedence operators (% and *) and then performs the addition.

id + id % id * id

+

/ \

id *

/ \

% id

/ \

id id

(iii) Show Derivation

Leftmost Derivation

E

$\Rightarrow E + T$

$\Rightarrow T + T$

$\Rightarrow F + T$

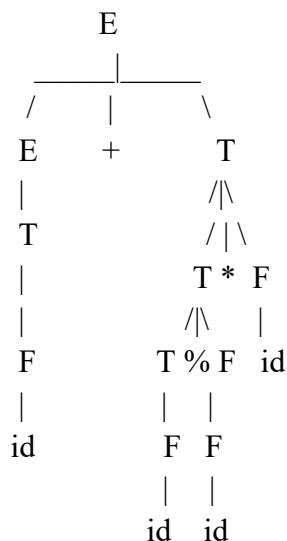
$\Rightarrow \text{id} + T$

$\Rightarrow id + T * F$
 $\Rightarrow id + T \% F * F$
 $\Rightarrow id + F \% F * F$
 $\Rightarrow id + id \% F * F$
 $\Rightarrow id + id \% id * F$
 $\Rightarrow id + id \% id * id$

Rightmost Derivation

E
 $\Rightarrow E + T$
 $\Rightarrow E + T * F$
 $\Rightarrow E + T * id$
 $\Rightarrow E + T \% F * id$
 $\Rightarrow E + T \% id * id$
 $\Rightarrow E + F \% id * id$
 $\Rightarrow E + id \% id * id$
 $\Rightarrow T + id \% id * id$
 $\Rightarrow F + id \% id * id$
 $\Rightarrow id + id \% id * id$

(iv) Construct Parse Tree



The parse tree shows that $\%$ and $*$ are evaluated before $+$.

(v) Discuss Ambiguity

The grammar is unambiguous.

Reason:

- The grammar separates operators into different precedence levels.
- $*$ and $\%$ are grouped under T , while $+$ is grouped under E .
- Every valid expression has a unique parse tree.

Therefore, the expression:

$id + id \% id * id$

has only one valid interpretation:

$id + ((id \% id) * id)$

Conclusion

The grammar correctly enforces operator precedence, where $*$ and $\%$ have higher precedence than $+$. The input is parsed uniquely, producing a single derivation and parse tree. Hence, the grammar is unambiguous and

suitable for compiler expression parsing.

3. A system parses file paths like: id/id/id

Grammar:

$P \rightarrow P / \text{id} \mid \text{id}$

Tasks:

- (i) Convert grammar to LL(1) form
- (ii) Demonstrate parsing
- (iii) Show derivation
- (iv) Construct parse tree
- (v) Explain ambiguity

Given Grammar

$P \rightarrow P / \text{id} \mid \text{id}$

Input

id/id/id

(i) Convert Grammar to LL(1) Form

The given grammar is left recursive, so it cannot be used directly by an LL(1) parser.

Original Grammar

$P \rightarrow P / \text{id} \mid \text{id}$

Remove Left Recursion

$P \rightarrow \text{id } P'$

$P' \rightarrow / \text{id } P' \mid \epsilon$

where ϵ (epsilon) represents the empty string.

This grammar is suitable for LL(1) parsing.

(ii) Demonstrate Parsing

Input:

id/id/id

Using the LL(1) grammar:

P

$\Rightarrow \text{id } P'$

$\Rightarrow \text{id} / \text{id } P'$

$\Rightarrow \text{id} / \text{id} / \text{id } P'$

$\Rightarrow \text{id} / \text{id} / \text{id}$

The parser successfully recognizes the file path.

(iii) Show Derivation

Leftmost Derivation

P

$\Rightarrow \text{id } P'$

$\Rightarrow \text{id} / \text{id } P'$

$\Rightarrow \text{id} / \text{id} / \text{id } P'$

$\Rightarrow \text{id} / \text{id} / \text{id } \epsilon$

$\Rightarrow \text{id} / \text{id} / \text{id}$

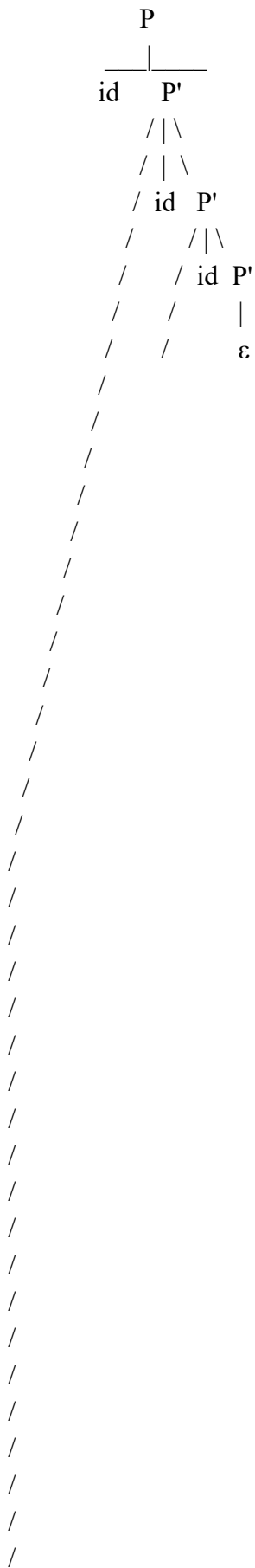
Rightmost Derivation

P

$\Rightarrow \text{id } P'$

$\Rightarrow \text{id} / \text{id } P'$
 $\Rightarrow \text{id} / \text{id} / \text{id } P'$
 $\Rightarrow \text{id} / \text{id} / \text{id } \epsilon$
 $\Rightarrow \text{id} / \text{id} / \text{id}$

(iv) Construct Parse Tree

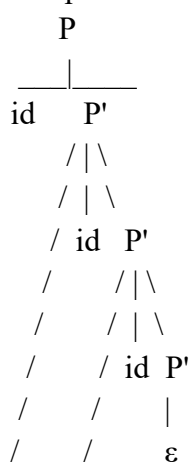


[illegible]

/ / /

$$\begin{array}{c} / \\ / \\ / \\ / \\ / \\ / \\ / \\ \mathbf{P'} \\ | \\ / \text{ id} \end{array}$$

A clearer representation:

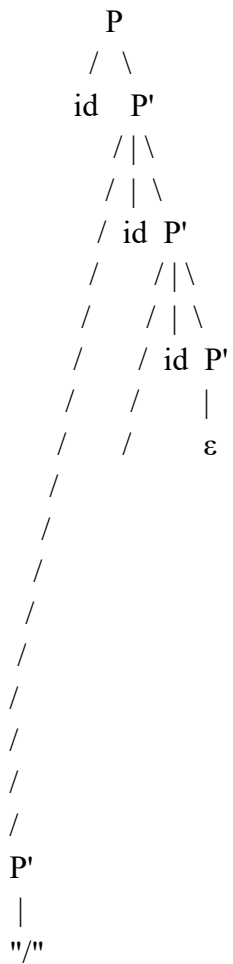


[illegible]

[illegible]

$$\begin{array}{c} / \\ / \\ / \\ / \\ / \\ / \\ / \\ \textbf{P}' \\ | \\ \boldsymbol{\varepsilon} \end{array}$$

Equivalent simple tree:



This parse tree represents the path:
id / id / id

(v) Explain Ambiguity

The grammar is unambiguous.

Reason:

- Every slash (/) connects exactly one identifier to the previous path.
- There is only one valid way to derive id/id/id.
- Therefore, only one parse tree is possible.

Hence, the grammar is not ambiguous.

Conclusion

- The original grammar is left recursive and is not suitable for LL(1) parsing.
- After removing left recursion:

$P \rightarrow id P'$

$P' \rightarrow / id P' \mid \varepsilon$

the grammar becomes LL(1).

- The input id/id/id is parsed successfully with a unique derivation and parse tree.
- Therefore, the grammar is unambiguous.

RUBRICS FOR CALCULATION:

Scenario based assessment

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand